

Projeto final - SOE

Esteira automatizada

Felipe de Castro Quixabeira
Faculdade de Ciências e Tecnologia
Universidade de Brasília

Paulo Caleb Fernandes da Silva
Faculdade de Ciências e Tecnologia
Universidade de Brasília

Resumo—Este trabalho apresenta o projeto e a implementação de um sistema embarcado para o controle de uma esteira automatizada destinada à separação de itens de pequeno porte. O sistema proposto é capaz de transportar objetos com restrições de massa e volume, realizando a identificação e o direcionamento em tempo real com base em critérios previamente definidos. A arquitetura integra uma unidade de controle de baixo nível baseada em microcontrolador, sensores para detecção e classificação de objetos e atuadores responsáveis pela separação em dois pontos de saída, além de um *Single Board Computer (SBC)* dedicado ao processamento de tarefas de alto nível em ambiente Linux embarcado.

I. INTRODUÇÃO

Sistemas de logística interna em centros de distribuição modernos (Fig. 1) dependem fortemente de linhas automatizadas para transporte e separação de itens, permitindo maior eficiência na preparação de pedidos e redução da intervenção humana. Empresas como Amazon e Mercado Livre utilizam sistemas altamente integrados, compostos por esteiras, sensores e robôs, capazes de identificar, rastrear e direcionar produtos em tempo real ao longo de múltiplos estágios do processo logístico [1], [2], [3], [4], [5]. A amplitude desse ecossistema pode ser observada em diversas referências audiovisuais [6], [7], [8], [9], e a tendência é que haja ampliação desses sistemas robotizados para suprir a crescente demanda por entregas e rastreamentos [5]. Nesse contexto, módulos de esteiras desempenham papel fundamental como unidades básicas de movimentação e roteamento de itens dentro de arquiteturas modulares e escaláveis.



Figura 1. Centro de distribuição do Mercado Livre em Cajamar, São Paulo.

O problema central abordado neste trabalho consiste no desenvolvimento de um módulo de esteira automatizada capaz

de reproduzir, em escala reduzida, o comportamento esperado de um sistema de separação de itens em uma linha de produção. Considera-se um fluxo contínuo de objetos, sendo necessário detectar sua presença, identificar o destino de cada objeto por meio da leitura de um código QR e direcioná-los corretamente para diferentes saídas. Esse problema envolve desafios típicos de sistemas embarcados, como controle em tempo real, integração entre sensores e atuadores, comunicação com sistemas externos e divisão eficiente de tarefas entre diferentes níveis de processamento.

No estado da arte, soluções industriais empregam arquiteturas distribuídas e altamente escaláveis, nas quais o processamento é dividido entre controladores de baixo nível, responsáveis pelo gerenciamento do sistema mecatrônico em tempo real, e sistemas de alto nível, responsáveis por visão computacional, tomada de decisão e integração com sistemas de gestão. No meio acadêmico, diversos trabalhos exploram sistemas de separação automatizada baseados em visão computacional, identificação por código de barras ou QR code e controle embarcado de esteiras, frequentemente utilizando plataformas como microcontroladores e computadores de placa única (*Single Board Computers* — SBCs) (Fig. 2) para prototipagem e validação experimental. Entretanto, muitas dessas abordagens focam isoladamente na identificação ou no controle, não explorando de forma integrada a comunicação eficiente entre camadas de processamento com restrições reais de *hardware*. Diante desse cenário, torna-se relevante o desenvolvimento de um protótipo acadêmico que permita explorar, de forma controlada, os principais desafios envolvidos na integração entre controle embarcado e processamento de alto nível, sem a complexidade e o custo associados a sistemas industriais completos.

Dessa forma, o presente trabalho propõe o desenvolvimento de um sistema embarcado para controle de um módulo de esteira automatizada em escala reduzida com uma entrada e duas saídas, capaz de realizar a separação de itens com base em decisões externas previamente associadas a um código QR de identificação do objeto. A arquitetura adotada diferencia-se por utilizar uma abordagem de processamento distribuído, na qual uma unidade de controle baseada em microcontrolador é responsável pelas tarefas de tempo real, enquanto um *Single Board Computer* executando Linux embarcado realiza tarefas de mais alto nível, como captura de imagem, leitura de código QR, definição de destino e supervisão do sistema.

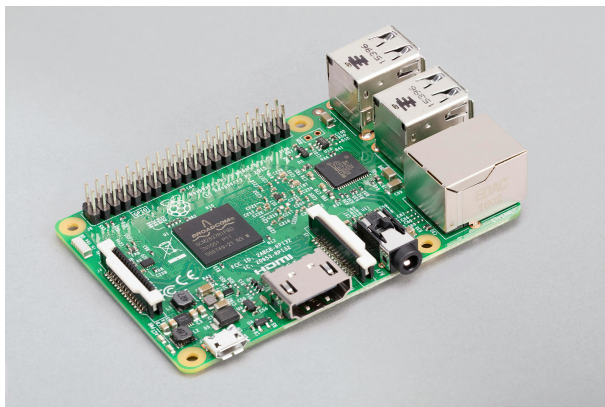


Figura 2. Raspberry Pi 3 Modelo B.

O sistema proposto tem como objetivo principal implementar um módulo funcional de separação de itens, capaz de detectar objetos, posicioná-los em uma zona de leitura, solicitar a identificação ao servidor Linux e executar o desvio para uma entre duas saídas possíveis. Para isso, são estabelecidos requisitos como a operação com objetos de pequeno porte (massa de até 250 g, volume máximo de 125 cm³ e altura entre 3 cm e 5 cm), o processamento sequencial de itens, a comunicação confiável entre o módulo embarcado e o servidor supervisor, e a execução determinística das ações de controle.

Como resultado esperado, busca-se a validação experimental de um protótipo funcional que demonstre a viabilidade da integração entre controle embarcado em tempo real e processamento de alto nível em ambiente Linux embarcado. A solução proposta também se destaca por seu caráter modular e didático, podendo servir como base para expansão em sistemas maiores de automação logística. Os arquivos do projeto estão disponíveis em repositório público no GitHub [10].

II. SOLUÇÃO PROPOSTA

A. Descrição de hardware

O sistema é composto por uma arquitetura híbrida que divide as responsabilidades entre um microprocessador Broadcom BCM2837, hospedado na placa Raspberry Pi 3B e responsável pelas tarefas de alto nível, e um microcontrolador STM32G070, dedicado ao controle em tempo real dos dispositivos físicos da esteira. O diagrama de blocos da Fig. 3 ilustra de forma simplificada a arquitetura do sistema.

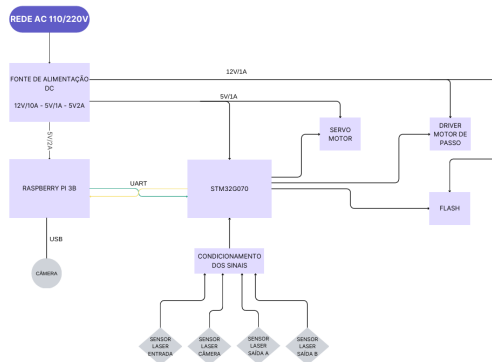


Figura 3. Diagrama de blocos do hardware.

A placa Raspberry Pi 3B foi escolhida devido à ampla documentação disponível, à acessibilidade ao hardware, uma vez que um dos integrantes da dupla já dispunha de uma unidade em funcionamento, e ao suporte nativo ao sistema operacional Linux, requisito para a execução da interface gráfica e dos algoritmos de visão computacional desenvolvidos neste projeto.

A câmera utilizada para a leitura e processamento de imagem é uma *webcam* genérica da marca Multilaser, com interface USB. A escolha foi motivada pela ausência de necessidade de *drivers* proprietários para o correto funcionamento com o Linux embarcado e pela disponibilidade imediata para o projeto.

O microcontrolador STM32G070 foi escolhido por sua variedade de periféricos adequados à aplicação, como comunicação UART, *timers* de uso geral e recurso de DMA (*Direct Memory Access*), que permite processar informações obtidas pelos periféricos sem ocupar a CPU. Além dessas características, o STM32G070 opera a uma frequência de CPU de 64 MHz e é suportado pelo ambiente de desenvolvimento integrado STM32CubeIDE, da STMicroelectronics, referência mundial na fabricação e suporte de microcontroladores para a indústria. O esquemático de conexão do microcontrolador pode ser observado na Fig. 4. Os *timers* são utilizados da seguinte forma:

- **TIM15:** gera o sinal PWM de frequência fixa (200 Hz, *duty cycle* de 50 %) utilizado para modular os lasers dos sensores;
- **TIM3:** configurado em modo *Input Capture* com DMA, realiza a medição de frequência dos quatro sensores laser simultaneamente;
- **TIM16:** gera o sinal PWM de 50 Hz com *duty cycle* variável para o controle do servo motor;
- **TIM6:** opera a 1 MHz e dispara interrupções periódicas para o controle preciso dos pulsos enviados ao *driver* do motor de passo.

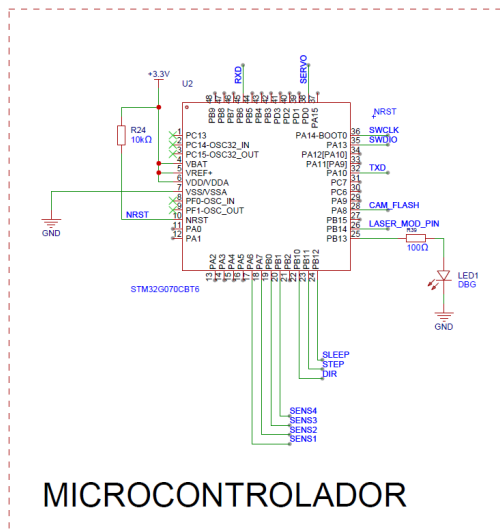


Figura 4. Esquemático do microcontrolador STM32G070.

A interface de programação e depuração do STM32G070 é realizada via *Serial Wire Debug* (SWD), conectada ao ST-Link V2 conforme o esquemático da Fig. 5. Os sinais SWCLK, SWDIO e NRST permitem gravar o *firmware* e depurar o sistema em tempo real diretamente pelo STM32CubeIDE.

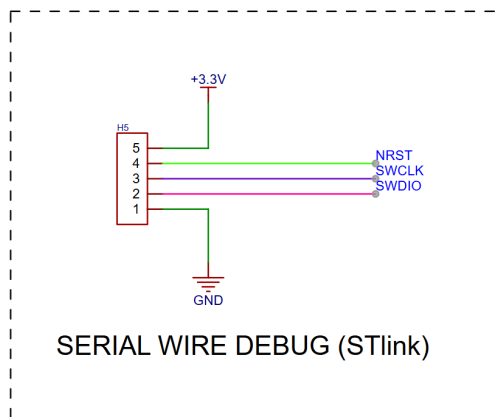


Figura 5. Esquemático da interface *Serial Wire Debug* (ST-Link).

A comunicação serial entre o STM32G070 e o Raspberry Pi 3B é realizada via UART, convertida para USB por meio de um módulo conversor dedicado, cujo esquemático é apresentado na Fig. 6. Esse conversor permite que o Raspberry Pi acesse a porta serial do microcontrolador como um dispositivo `/dev/ttyUSB`, sem necessidade de adaptação de nível lógico adicional, uma vez que tanto o STM32 quanto o conversor operam em 3,3 V.

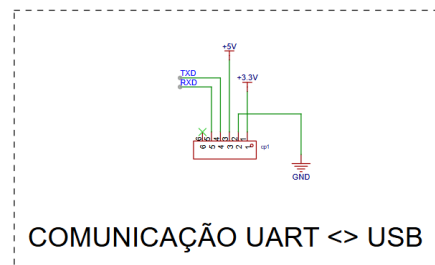


Figura 6. Esquemático do conversor UART $\leftrightarrow$ USB.

A alimentação do sistema é gerenciada por dois estágios de regulação, conforme o esquemático da Fig. 7. A tensão de entrada de 12 V, proveniente de uma fonte externa, é convertida para 5 V pelo módulo *buck* LM2596S, protegido por um diodo 6A10 contra inversão de polaridade. O regulador linear AMS1117-3.3 deriva os 3,3 V necessários para a alimentação do STM32G070 e dos circuitos de sinal a partir dos 5 V. Capacitores de desacoplamento de 100 nF estão presentes na entrada e saída do AMS1117-3.3 para rejeição de ruído de alta frequência.

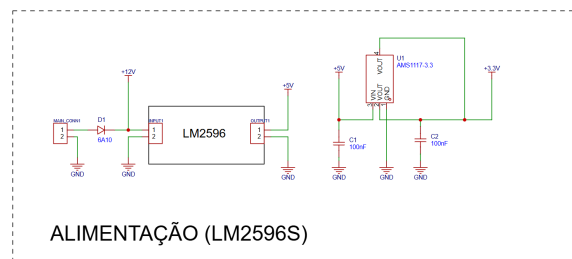


Figura 7. Esquemático do circuito de alimentação (LM2596S + AMS1117-3.3).

Como sistema motor do tapete da esteira, foi utilizado um motor de passo NEMA 22 controlado pelo *driver* A4988, conforme o esquemático da Fig. 8. Os pinos STEP, DIR e SLEEP são utilizados, respectivamente, para avançar um passo, definir a direção de rotação e habilitar ou desabilitar o *driver* (liberando ou travando o eixo). Esses componentes foram escolhidos por sua disponibilidade e pela baixa complexidade operacional do *driver*.

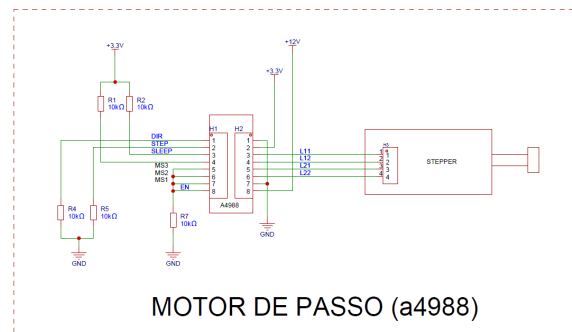


Figura 8. Esquemático do motor de passo e do *driver* A4988.

Como atuador da cancela, foi utilizado o servo motor ES08MA (Fig. 9), selecionado pela disponibilidade para o projeto e pelo torque superior ao do modelo SG90. O servo é controlado por PWM de 50 Hz, com largura de pulso variando entre 600 μ s (0°) e 2300 μ s (180°), mapeada diretamente a partir do ângulo desejado pelo *firmware*.

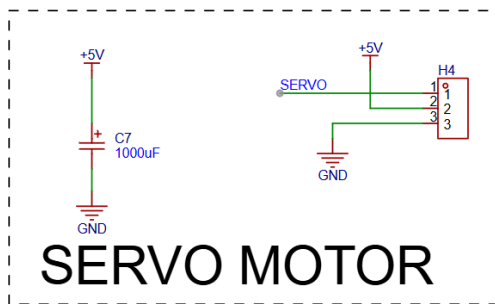


Figura 9. Esquemático do servo motor ES08MA.

Os sensores laser foram projetados para apresentar a maior simplicidade possível e para que o circuito eletrônico fosse viável de fabricação artesanal em *breakout boards* com solda manual. O princípio de funcionamento é baseado em detecção síncrona por ausência de portadora: o laser modula seu feixe na frequência gerada pelo TIM15 (200 Hz). O feixe incide sobre um LDR que, após condicionamento de sinal com amplificadores operacionais (LM324), entrega ao STM32G070 um sinal digital pulsado na faixa de 0 a 3,3 V com frequência idêntica à portadora. Quando um objeto interrompe o feixe, a ausência do sinal pulsado, interpretada pelo *firmware* como nível lógico alto permanente na entrada de captura, é identificada como presença de objeto à frente do sensor. O circuito conta com quatro instâncias desse sensor, dispostas em posições estratégicas ao longo da esteira: entrada da esteira (S1), zona de leitura do QR (S2), saída da Rota A (S3) e saída da Rota B (S4). O esquemático completo dos quatro sensores é apresentado na Fig. 10.

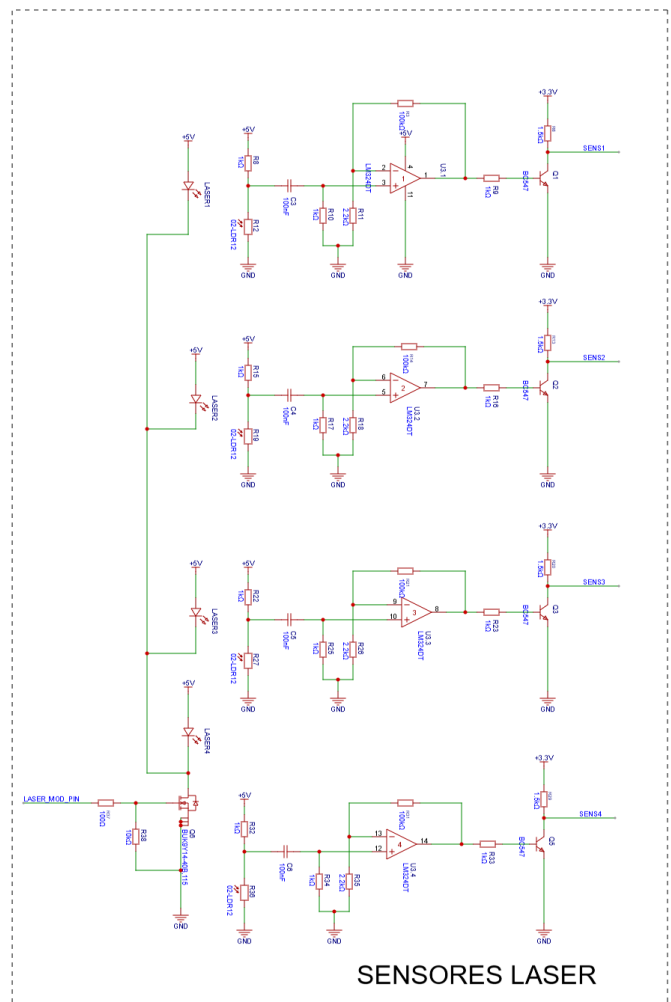


Figura 10. Esquemático dos quatro sensores laser com condicionamento de sinal (LDR + LM324).

Para auxiliar a visualização do código QR pela câmera, o sistema conta com uma luminária composta por seis LEDs de uma fita LED 4000 K. A temperatura de cor foi escolhida por apresentar maior similaridade com a luz ambiente, o que resultou em melhor qualidade de captura da câmera em comparação com fitas de 6500 K. O acionamento da luminária é realizado por uma GPIO dedicada do STM32G070, por meio de um MOSFET BUK9Y14 em configuração de chave, como ilustrado no esquemático da Fig. 11.

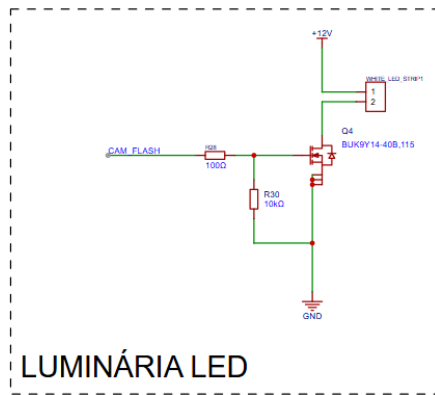


Figura 11. Esquemático do circuito de acionamento da luminária LED.

O esquemático completo da placa de circuito montada para o projeto encontra-se disponível no repositório do projeto [10].

Para a montagem completa do sistema, foi elaborada a lista de materiais apresentada nas Tabelas I e II.

Tabela I
LISTA DE MATERIAIS — ELETRÔNICA.

Componente	Preço unit. (R\$)	Qtd.
Raspberry Pi 3B	—	1
Webcam genérica	—	1
ST-Link V2	—	1
STM32G070	5,50	1
Driver A4988	18,00	1
AMS1117-3.3	1,00	1
Módulo Buck LM2596S	15,00	1
AmpOp LM324	3,50	1
Diodo 1N4007	0,50	1
Capacitor eletrolítico 1000 µF	0,50	1
Motor de passo NEMA 22	60,00	1
Servo motor ES08MA	25,00	1
MOSFET BUK9Y14	1,00	2
Transistor BJT BC547B	0,20	4
Resistor 100 Ω	0,10	3
Resistor 1 kΩ	0,10	12
Resistor 1k5 Ω	0,10	4
Resistor 2k2 Ω	0,10	4
Resistor 10 kΩ	0,10	8
Resistor 100 kΩ	0,10	4
LDR 10 kΩ	0,50	4
Laser com ajuste focal	1,00	4
Capacitor 100 nF	0,10	6
Fio coaxial (sensores LDR)	10,00	2 m
Fio 22 AWG vermelho (lasers)	1,50	2 m
Fio 22 AWG azul (lasers)	1,50	2 m
Fio 22 AWG preto (servo)	1,50	1 m
Fio 22 AWG branco (servo)	1,50	1 m
Fio 22 AWG amarelo (servo)	1,50	1 m
Conector Molex KK macho 2 terminais	0,10	8
Conector Molex KK macho 3 terminais	0,15	2
Conector Molex KK fêmea 2 terminais	0,10	8
Conector Molex KK fêmea 3 terminais	0,15	2
Total (itens com preço definido)	163,80	

Tabela II
LISTA DE MATERIAIS — MECÂNICA.

Componente	Preço unit. (R\$)	Qtd.
Perfil de alumínio 20×20 mm	—	—
Chapa de PVC (rampa saída B)	—	1
Tapete de PVC	—	1
Rolo motriz (impressão 3D)	—	1
Rolo bobo (impressão 3D)	—	1
Lateral esq. mesa esteira (imp. 3D)	—	1
Lateral dir. mesa esteira (imp. 3D)	—	1
Lateral esq. rampa (imp. 3D)	—	1
Lateral dir. rampa (imp. 3D)	—	1
Cancela servo (imp. 3D)	—	1
Suporte servo (imp. 3D)	—	1
Suporte ST-Link (imp. 3D)	—	1
Suporte laser c/ ajuste focal (imp. 3D)	—	3
Suporte LDR (imp. 3D)	—	3
Suporte laser rampa (imp. 3D)	—	1
Suporte LDR rampa (imp. 3D)	—	1
Suporte flash esteira (imp. 3D)	—	2
Suporte poste câmera (imp. 3D)	—	2
Suporte câmera esteira (imp. 3D)	—	1

B. Descrição de software

O *software* do sistema é dividido em dois níveis de processamento: o *firmware* embarcado no STM32G070, responsável pelo controle em tempo real dos atuadores e sensores, e a interface gráfica executada no Raspberry Pi 3B, responsável pela supervisão do sistema, leitura de código QR e comunicação com o microcontrolador.

1) *Firmware do STM32G070*: O *firmware* foi desenvolvido em linguagem C utilizando os *drivers* HAL da STMicroelectronics no ambiente STM32CubeIDE. Sua estrutura principal é composta por uma rotina de inicialização, um laço principal (*loop* infinito) e uma Máquina de Estados Finitos (FSM).

a) *Rotina de inicialização*.: Antes de entrar no laço principal, o *firmware* executa uma rotina de bloqueio que aguarda simultaneamente duas condições: (i) todos os quatro sensores laser devem operar na frequência esperada de 200 Hz, e (ii) o Raspberry Pi deve enviar o byte de *handshake* SYS_RDY (0x10). Durante esse período, um LED de depuração indica o estado do sistema: pisca a 100 ms se os sensores ainda não estiverem prontos, e a 500 ms quando aguarda apenas o *handshake*. Ao receber o sinal do Raspberry Pi, o STM32G070 responde com SYS_INIT (0x01) e ingressa no laço principal.

b) *Leitura dos sensores laser*.: A leitura dos sensores é baseada em detecção síncrona por ausência de portadora. O TIM15 gera um PWM de 200 Hz para modular os quatro lasers simultaneamente. O TIM3, em modo *Input Capture* com DMA, mede a frequência do sinal retornado por cada LDR. Se a frequência capturada corresponde à portadora (200 Hz), o feixe está livre; se a captura retorna zero (ausência de sinal), há um objeto interrompendo o feixe. O laço principal chama `getSensorsReadings()` a cada intervalo definido, atualizando as variáveis de estado `SENSn_freq` e `SENSn_flag` para cada sensor $n \in \{1, 2, 3, 4\}$.

c) *Controle do motor de passo*.: O motor de passo NEMA 22 é acionado via TIM6, que opera a 1 MHz e gera interrupções periódicas para o controle preciso dos

pulsos enviados ao pino STEP do *driver* A4988. O sentido de rotação é determinado pelo pino DIR, e o pino SLEEP controla o engajamento do eixo. Funções auxiliares como `stepperMove()`, `stepperStopEngaged()`, `stepperSetSpeed()`, e `stepperStopDisengaged()` encapsulam o controle do motor, permitindo que a FSM abstraia os detalhes de temporização.

d) *Controle do servo motor*.: O servo ES08MA é controlado pelo TIM16, que gera um PWM de 50 Hz com *duty cycle* variável. A função `SetServoAngle(angle)` mapeia o ângulo desejado (em graus) para o valor de comparação do registrador do *timer*, com largura de pulso variando de 600 µs (0°) a 2300 µs (180°). No contexto da FSM, a cancela opera em dois estados: fechada (0°), posicionando os objetos para a Rota A, e aberta (45°), desviando-os para a Rota B.

e) *Máquina de Estados Finitos (FSM)*.: O fluxo de operação do sistema é controlado por uma FSM implementada na função `FSM()`, chamada ciclicamente no laço principal. O diagrama de estados é apresentado na Fig. 12.

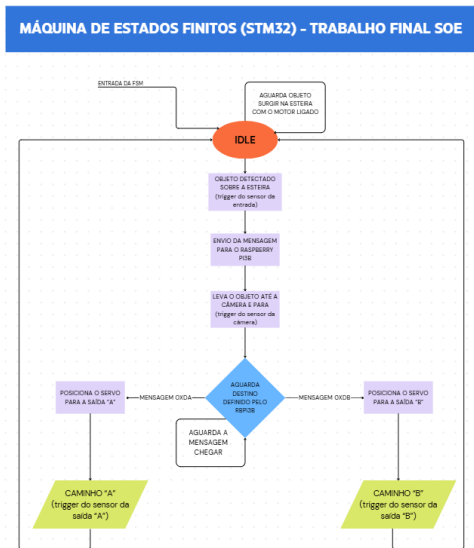


Figura 12. Diagrama de estados da FSM do *firmware* do STM32G070.

Os estados da máquina são:

- **IDLE**: sistema aguardando objeto. Motor ligado, cancela fechada, *flash* apagado;
- **OBJ_DETECTED**: objeto detectado pelo sensor S1. Motor continua em movimento transportando o objeto até a zona de leitura;
- **CLASSIF_REQUEST**: objeto parado sob o sensor S2 (zona de leitura). Motor para. *Flash* aceso. STM32 envia `CLSS_REQUEST` (0xC0) ao Raspberry Pi e aguarda resposta de roteamento;
- **ROUTE_A**: Raspberry Pi enviou `ROUTE_A` (0xDA). Cancela permanece fechada. Motor reinicia. STM32 envia `ROUTE_A_FWRDNG` (0xFA) e aguarda detecção no sensor S3;
- **ROUTE_B**: Raspberry Pi enviou `ROUTE_B` (0xDB). Cancela abre (45°). Motor reinicia. STM32 envia

`ROUTE_B_FWRDNG` (0xFB) e aguarda detecção no sensor S4;

- **DELIVERED**: sensor de saída detectou o objeto. Mensagem de entrega bem-sucedida enviada ao Raspberry Pi. Sistema retorna a IDLE.

f) *Modo de depuração (DEBUG)*.: O *firmware* implementa um modo de operação alternativo, ativado pelo comando `DEBUG_TOGGLE` (0xDD). Nesse modo, a FSM é suspensa e o STM32G070 passa a responder exclusivamente a comandos assíncronos enviados pelo Raspberry Pi, permitindo o acionamento individual de cada atuador (*flash*, cancela, motor de passo) para fins de diagnóstico e validação. O STM32 também envia espontaneamente a telemetria dos quatro sensores (`SENS_STATUS`, 0x55) sempre que o estado de algum sensor muda.

g) *Protocolo de comunicação UART*.: A comunicação entre o STM32G070 e o Raspberry Pi 3B segue um protocolo proprietário desenvolvido para este projeto, operando a 115200 bps (8N1 - oito bits de dados, sem paridade, um stop bit). O recebimento é gerenciado por interrupção (`HAL_UART_Receive_IT`), garantindo que nenhum byte seja perdido durante a execução da FSM. A estrutura dos quadros é definida conforme a Tabela III.

A Tabela IV apresenta as principais mensagens definidas no protocolo, incluindo o *handshake* de inicialização, as mensagens da FSM e os comandos de controle assíncrono.

Tabela III
ESTRUTURA DOS QUADROS UART.

Direção	Formato
RPi → STM32 (RX)	[0xAA] [CMD] ou [0xAA] [0xE5] [DATA]
STM32 → RPi (TX ok)	[0x90] [PAYLOAD]
STM32 → RPi (TX erro)	[0x91]

Tabela IV
MENSAGENS DO PROTOCOLO UART PROPRIETÁRIO.

Nome	Descrição	Valor	Dir.
<i>Handshake</i>			
<code>SYS_RDY</code>	RPi sinaliza	0x10	RX
<code>SYS_INIT</code>	STM32 confirma	0x01	TX
<i>Mensagens da FSM</i>			
<code>OBJ_DETECTED</code>	Objeto detectado (S1)	0xA0	TX
<code>CLSS_REQUEST</code>	Solicita classificação	0xC0	TX
<code>ROUTE_A</code>	Ordena rota A	0xDA	RX
<code>ROUTE_B</code>	Ordena rota B	0xDB	RX
<code>ROUTE_A_FWRDNG</code>	Confirm. encaminh. A	0xFA	TX
<code>ROUTE_B_FWRDNG</code>	Confirm. encaminh. B	0xFB	TX
<code>ROUTE_A_OK</code>	Entrega concluída A	0xBA	TX
<code>ROUTE_B_OK</code>	Entrega concluída B	0xBB	TX
<i>Controle assíncrono (modo DEBUG)</i>			
<code>LIGHT_EN</code>	Liga luminária	0xE1	RX
<code>LIGHT_DIS</code>	Desliga luminária	0xD1	RX
<code>GATE_OPEN</code>	Abre cancela (45°)	0xE2	RX
<code>GATE_CLOSE</code>	Fecha cancela (0°)	0xD2	RX
<code>STPR_EN</code>	Engaja motor	0xE3	RX
<code>STPR_DIS</code>	Libera motor	0xD3	RX
<code>STPR_FWD</code>	Rotação: frente	0xE4	RX
<code>STPR_BCK</code>	Rotação: trás	0xD4	RX
<code>STPR_TGT_STPS</code>	Define N passos	0xE5	RX
<code>SENS_STATUS</code>	Telemetria sensores	0x55	TX

h) *Gerador de códigos QR*.: Para a criação dos códigos QR utilizados nos testes, foi desenvolvido um script Python

auxiliar (`gerar_qrcodes.py`) que lê um arquivo JSON com as informações das peças e gera as imagens correspondentes. Cada código QR codifica o nome da peça, o destino (endereço hexadecimal utilizado pela FSM) e a frase de identificação do projeto, conforme o formato arbitrado pela dupla.

III. RESULTADOS EXPERIMENTAIS

Os experimentos descritos nesta seção foram realizados com o objetivo de validar o funcionamento de cada subsistema fundamental da esteira, bem como a integração entre o microcontrolador STM32G070 e o Raspberry Pi 3B. Para cada subsistema, descreve-se o procedimento adotado, os resultados obtidos e, quando pertinente, as divergências observadas e suas possíveis causas.

a) *Sensores laser*.: Os quatro sensores operam conforme especificado. A lógica de detecção síncrona por ausência de portadora foi validada com sucesso — ao interromper o feixe, as *flags* internas (`SENS1_flag` a `SENS4_flag`) são acionadas corretamente, permitindo as transições de estado da FSM.

b) *Flash (luminária)*.: O controle via GPIO dedicada funciona corretamente. O *flash* é ativado e desativado em resposta aos comandos assíncronos recebidos via UART, sem instabilidades observadas.

c) *Motor de passo*.: O motor opera de forma estável em frequências mais baixas de acionamento. Os testes indicaram que o controle de velocidade por alteração do ARR do TIM6 funciona, porém com uma faixa operacional limitada — comportamento esperado dadas as características físicas do motor de passo (torque *versus* velocidade). Em razão disso, a velocidade de operação será mantida fixa no valor mínimo configurado (`STEPPER_MIN_VEL` = 300 passos/s), que corresponde a um período de interrupção do TIM6 de aproximadamente 1,67 ms, calculado com base na configuração de *prescaler* de 63 e *clock* de 64 MHz.

d) *Servo motor*.: O servo motor ES08MA responde corretamente a qualquer valor de angulação dentro da faixa suportada (0° a 180°). Para a aplicação na esteira, os ângulos definidos no *firmware* — 0° para cancela fechada (Rota A) e 45° para cancela aberta (Rota B) — foram validados e apresentaram o posicionamento mecânico desejado.

REFERÊNCIAS

- [1] Engenharia Híbrida, “Conheça o processo inovador de gestão de estoques da Amazon,” Disponível em <https://www.engenhariahibrida.com.br/post/conheca-processo-inovador-gestao-de-estoques-amazon> (30/03/2026).
- [2] Equipe TOTVS, “Logística do Mercado Livre: entenda os pilares da eficiência de distribuição da empresa,” Disponível em <https://www.totvs.com/blog/gestao-logistica/logistica-mercado-livre/> (30/03/2026), 12 2023.
- [3] —, “Logística da Amazon: entenda os pilares da empresa líder do varejo mundial,” Disponível em <https://www.totvs.com/blog/gestao-logistica/logistica-amazon/> (30/03/2026), 04 2024.
- [4] Mercado Livre, “Mercado Envios Full,” Disponível em <https://www.mercadolivre.com.br/l/envios-full> (30/03/2026).
- [5] Amazon Staff, “Amazon unveils the next generation of fulfillment centers powered by AI and 10 times more robotics,” Disponível em <https://www.aboutamazon.com/news/operations/amazon-fulfillment-center-robotics-ai> (30/03/2026).
- [6] YouTube, “Vídeo – referência de automação logística,” Disponível em <https://www.youtube.com/watch?v=cRC4iktG-kE> (30/03/2026).
- [7] —, “Vídeo – escaneamento e automação de itens em esteira,” Disponível em <https://youtu.be/iuMmexYY1hg?t=111> (30/03/2026).
- [8] —, “Vídeo – sistema de separação e desvio de itens,” Disponível em <https://www.youtube.com/watch?v=7AHLNGUSeGQ&t=8s> (30/03/2026).
- [9] Instagram, “Reel – demonstração de sistema automatizado de esteira,” Disponível em <https://www.instagram.com/reels/DQKF8Hbja6q/> (30/03/2026).
- [10] Caleb P. e Castro F., “Trabalho final SOE 2026.1,” Disponível em https://github.com/pauloCaleb/Trabalho_Final_SOE_2026.1_Esteira.git (30/03/2026).

APÊNDICE

Os documentos a seguir estão anexados nas páginas subsequentes e encontram-se disponíveis no repositório do projeto [10], servem como documentação local do repositório e estão sendo apresentados como apêndices a esse trabalho:

- **Anexo A — Fechamento do Escopo** — documento que concentra de forma sucinta o contexto de aplicação, os objetivos, as limitações e as referências bibliográficas do projeto.
- **Anexo B — Descrição das Peças** — descrição de cada peça mecânica fabricada por impressão 3D para a esteira, incluindo função e quantidade.
- **Anexo C — Tabela de Mensagens STM32G070** — tabela completa do protocolo UART, com os fluxos de *handshake*, FSM e controle assíncrono, bytes hexadecimais e ações correspondentes.

Fechamento do Escopo – Projeto Final SOE

1. Contexto de Aplicação

Sistemas de logística interna em centros de distribuição utilizam linhas automatizadas de transporte e separação de itens para preparação de pedidos. Esses sistemas são compostos por módulos de esteiras interconectados, capazes de identificar e direcionar produtos conforme seu destino.

2. Definição do Problema

Dado um fluxo contínuo de itens previamente identificados, é necessário desenvolver um módulo de esteira automatizado capaz de:

- Permitir a identificação de cada item;
 - Determinar o destino de cada item;
 - Direcioná-lo corretamente dentro de uma linha de separação modular.
-

3. Objetivo do Projeto

Desenvolver um sistema embarcado para controle de um módulo de esteira automatizada com capacidade de identificação e desvio de itens de acordo com o destino previamente associado ao objeto, com ênfase na integração entre controle embarcado e processamento em sistema Linux embarcado.

Objetivos Específicos

- Implementar controle de movimentação da esteira;
 - Desenvolver mecanismo de desvio de itens;
 - Integrar comunicação com sistema externo;
 - Validar o funcionamento do sistema experimentalmente;
-

4. Escopo do Sistema

O sistema consiste em um módulo de esteira com:

- 1 entrada de itens;
- 2 saídas (desvio local ou encaminhamento para próximo módulo);

Capaz de operar com objetos que atendam às seguintes restrições:

- Massa máxima: 250 g
 - Volume máximo: 125 cm³
 - Altura: entre 2 cm e 5 cm
-

5. Funcionalidades do Sistema

O sistema embarcado deverá:

1. Detectar a presença de objetos na esteira;
 2. Controlar o deslocamento do item até a zona de leitura;
 3. Solicitar ao servidor, via interface de comunicação, a identificação do item;
 4. Receber do servidor a decisão de destino;
 5. Executar o desvio do item para uma das saídas;
-

6. Interface com o servidor (ex.: Raspberry Pi)

O módulo deverá possuir interface de comunicação com um servidor que será responsável por:

- Captura de imagem do item na zona de leitura;
 - Processamento e decodificação do código QR;
 - Determinação do destino do item;
 - Envio de comandos ao módulo da esteira;
 - Monitoramento do estado do sistema;
 - Registro de eventos de operação;
-

7. Limitações do Projeto

Este projeto não contempla:

- Integração com sistemas reais de estoque;
 - Implementação de múltiplos módulos interconectados;
 - Processamento de múltiplos itens simultaneamente;
 - Sistemas avançados de visão computacional.
-

8. Resultados Esperados

- Protótipo funcional de um módulo de esteira automatizado validado experimentalmente;

- Sistema embarcado operando integrado a um ambiente Linux embarcado;
 - Capacidade de identificação e separação de itens conforme o destino definido;
 - Documentação técnica completa do sistema;
-

9. Aplicação

O sistema proposto se aplica a processos de automação logística em centros de distribuição, podendo ser utilizado como base para sistemas modulares escaláveis de separação de pedidos. A arquitetura proposta segue o modelo de processamento distribuído, separando tarefas de tempo real e processamento de alto nível.

10. Referências Bibliográficas

- [1] ENGENHARIA HÍBRIDA. Conheça o processo inovador de gestão de estoques da Amazon. Engenharia Híbrida, [s.d.]. Disponível em: <https://www.engenhariahibrida.com.br/post/conheca-processo-inovador-gestao-de-estoques-amazon>. Acesso em: 30 mar. 2026.
- [2] EQUIPE TOTVS. Logística do Mercado Livre: entenda os pilares da eficiência de distribuição da empresa. TOTVS Blog, 25 dez. 2023. Disponível em: <https://www.totvs.com/blog/gestao-logistica/logistica-mercado-livre/>. Acesso em: 30 mar. 2026.
- [3] EQUIPE TOTVS. Logística da Amazon: entenda os pilares da empresa líder do varejo mundial. TOTVS Blog, 16 abr. 2024. Disponível em: <https://www.totvs.com/blog/gestao-logistica/logistica-amazon/>. Acesso em: 30 mar. 2026.
- [4] MERCADO LIVRE. Mercado Envios Full. Mercado Livre, [s.d.]. Disponível em: <https://www.mercadolivre.com.br/l/envios-full>. Acesso em: 30 mar. 2026.
- [5] AMAZON STAFF. Amazon unveils the next generation of fulfillment centers powered by AI and 10 times more robotics. About Amazon, [s.d.]. Disponível em: <https://www.aboutamazon.com/news/operations/amazon-fulfillment-center-robotics-ai>. Acesso em: 30 mar. 2026.
- [6] YOUTUBE. [Vídeo – referência de automação logística]. Disponível em: <https://www.youtube.com/watch?v=cRC4iktG-kE>. Acesso em: 30 mar. 2026.
- [7] YOUTUBE. [Vídeo – escaneamento e automação de itens em esteira]. Disponível em: <https://youtu.be/iumMexYY1hg?t=111>. Acesso em: 30 mar. 2026.
- [8] YOUTUBE. [Vídeo – sistema de separação e desvio de itens]. Disponível em: <https://www.youtube.com/watch?v=7AHLNGUSeGQ&t=8s>. Acesso em: 30 mar. 2026.

[9] INSTAGRAM. [Reel – demonstração de sistema automatizado de esteira]. Disponível em: <https://www.instagram.com/reels/DQKF8Hbja6g/>. Acesso em: 30 mar. 2026.

Descrição das Peças Mecânicas – Esteira Separadora SOE

Peças do Subsistema Estrutural

Autor: Paulo Caleb Fernandes da Silva

Wood_stitches_Esteira_Rolante_SOE.catPART

Quantidade: 3 unidades

Peças plásticas retangulares com dois furos espaçados para fixar duas partes de madeira que compõem a base da esteira. Funcionam como "pontos" que seguram uma madeira na outra e conferem rigidez à base.

Apoio_Base_Esteira_Rolante_SOE.catPART

Quantidade: 8 unidades

Bases de apoio retangulares com quatro furos para parafusos e um ressalto para servir de pé para a esteira. São utilizadas para duas funções simultaneamente: complementam a costura das peças de madeira e servem de espaçadores da base em relação ao solo.

Suporte_Motor_Esteira_Rolante_SOE.catPART

Quantidade: 1 unidade

Base de fixação do motor de passo para garantir estabilidade mecânica do motor NEMA 22 e alinhá-lo com o suporte do rolo.

Trava_Suporte_Motor_Esteira_Rolante_SOE.catPART

Quantidade: 1 unidade

Aro de fixação do motor de passo da esteira. É responsável por auxiliar a fixação do motor em conjunto com o `Suporte_Motor_Esteira_Rolante_SOE`. As duas peças trabalham juntas para manter a fixação do motor de passo.

`Suporte_Parafuso_Esteira_Rolante_SOE.catPART`

Quantidade: 3 unidades

Suporte do parafuso que faz o alinhamento do rolo da esteira. O parafuso é fixado ao suporte e cria um ponto de eixo para o rolamento do rolo. É utilizado tanto para o rolo motriz (associado ao motor de passo) quanto para o rolo bobo (utilizando dois suportes para criar os dois pontos de eixo do rolo).

`Tampa_Rolo_Motor_Esteira_Rolante_SOE.catPART`

Quantidade: 1 unidade

Tampa do rolo motriz, que associa o eixo do motor de passo à estrutura do rolo. Em conjunto com a `Tampa_Rolo_Rolamento_Esteira_Rolante_SOE` e um cano de PVC de 50 mm, forma o rolo motriz da esteira.

`Tampa_Rolo_Rolamento_esteira_rolante_SOE.catPART`

Quantidade: 3 unidades

Tampa do rolo bobo, que associa o eixo do parafuso à estrutura do rolo. O rolo motriz utiliza uma unidade desta peça, enquanto o rolo bobo utiliza duas. A fixação do rolo bobo emprega dois `Suporte_Parafuso_Esteira_Rolante_SOE` em conjunto com dois rolamentos 608 que se apoiam nos parafusos fixados aos suportes.

`Suporte_mesa_Rolante_SOE.catPART`

Quantidade: 6 unidades

Suporte para a chapa de PVC que forma a mesa da esteira. Um conjunto dessas peças forma a base da mesa por onde corre o tapete, guiado pelo motor de passo a partir do rolo motriz e do rolo bobo.

`Lateral_esq_rampa_esteira_rolante_SOE.catPART`

Quantidade: 1 unidade

Lateral esquerda com alojamento para encaixe de uma chapa de PVC que forma a rampa da saída B da esteira.

Lateral_dir_rampa_esteira_rolante_SOE.catPART

Quantidade: 1 unidade

Lateral direita com alojamento para encaixe de uma chapa de PVC que forma a rampa da saída B da esteira.

Peças do Subsistema de Sensoriamento, Atuação e Eletrônica

Autor: Felipe de Castro

Cancela_servo.catPART

Quantidade: 1 unidade

Estrutura fixada ao eixo do servo motor responsável por guiar o objeto sobre a esteira para a saída B.

Suporte_Servo.catPART

Quantidade: 1 unidade

Peça de fixação do servo motor. Este suporte é colado sobre a mesa da esteira e posiciona o servo motor na altura e orientação necessárias para suprir as demandas do escopo do projeto.

Suporte_STlink.catPART

Quantidade: 1 unidade

Suporte para fixação do ST-Link (programador e debugger do STM32) no corpo da esteira.

Suporte_laser_c_ajuste_focal.catPART

Quantidade: 3 unidades

Suporte do laser com ajuste focal para o sensor laser. Serve para alinhar o laser ao LDR e deve ser fixado na mesa da esteira com bom alinhamento entre os dois dispositivos. Cada sensor disposto na mesa da esteira possui um suporte de laser.

Suporte_LDR.catPART

Quantidade: 3 unidades

Suporte do LDR. Serve para alinhar o LDR ao laser e deve ser fixado na mesa da esteira com bom alinhamento entre os dois dispositivos. Cada sensor na mesa possui um suporte de LDR.

Suporte_laser_Rampa.catPART

Quantidade: 1 unidade

Suporte do laser para a rampa. É colado à lateral da rampa e cria um ponto de fixação por parafuso que prende o conjunto à base da esteira. Deve ser colado alinhado ao suporte do LDR da rampa.

Suporte_LDR_sensor_Rampa.catPART

Quantidade: 1 unidade

Suporte do LDR para a rampa. É colado à lateral da rampa e cria um ponto de fixação por parafuso que prende o conjunto à base da esteira. Deve ser colado alinhado ao suporte do laser da rampa.

Suporte_flash_esteira.catPART

Quantidade: 2 unidades

Suporte para a fita de LED que ilumina o objeto na região de leitura do código QR.

Suporte_poste_camera.catPART

Quantidade: 2 unidades

Suporte plástico que une os dois suportes do flash e forma a estrutura da câmera, fechando a região de leitura da esteira. São utilizadas duas peças para separar igualmente os dois postes do flash.

Quantidade: 1 unidade

Ponto de fixação da câmera. Deve ser colado sobre os suportes que formam a região de leitura de forma que a câmera fique alinhada com o sensor laser onde o objeto é posicionado sobre o tapete da esteira.

Case de fixação do Raspberry Pi 3b

Autor: Nilo - Usuário do MakerWorld

O case de fixação do RBPi3B utilizado foi obtido através da plataforma Open Source *MakerWorld*, que é um site onde usuários da comunidade de impressão 3D compartilham seus próprios desenhos de forma aberta sem fim comercial.

A case foi desenhada inicialmente para o RBPi4 e após impressa foi modificada para abrir os alojamentos das portas USB e HDMI.

O case original pode ser encontrado no seguinte link <https://makerworld.com/pt/models/1500557-case-for-raspberry-pi-4-with-40mm-fan?from=search#profileId-1569716>

Tabela de Mensagens – Firmware STM32G070

v3

Protocolo de quadro:

- **RX** (Raspberry Pi → STM32): `[0xAA] [CMD]` ou `[0xAA] [CMD] [DATA]`
- **TX** (STM32 → Raspberry Pi): `[0x90] [PAYLOAD]` (sucesso), `[0x90] [CMD] [DATA]` (telemetria), ou `[0x91]` (erro)

Tabela 1 – Dicionário Completo de Mensagens

Símbolo	Byte	Direção	Tipo	Descrição
START FRAME _MSG	0x AA	RX	Estrutura	Byte de início de quadro. Todo frame recebido deve começar com este byte; bytes fora de quadro são ignorados silenciosamente.
CMD O K_MSG	0x 90	TX	Estrutura	Byte de status positivo. Enviado como primeiro byte de qualquer resposta bem-sucedida, sempre seguido de um payload.
CMD E RR_MS G	0x 91	TX	Estrutura	Byte de status negativo. Enviado sozinho (1 byte) quando o comando recebido não é reconhecido.
SYS R DY_MS G	0x 10	RX	Handshake	Mensagem enviada pelo Raspberry Pi para iniciar o handshake. Só é aceita após todos os quatro sensores estarem operando na frequência esperada (200 Hz).
SYS I NIT_M SG	0x 01	TX	Handshake	Payload de resposta ao handshake. Indica que o STM32 está inicializado. Enviado no quadro <code>[0x90] [0x01]</code> , sempre seguido de <code>[0x90] [0x11]</code> informando o modo inicial.
MODE_ FSM_M SG	0x 11	TX	Modo	Payload que informa que o modo ativo é <code>OP_MODE_FSM</code> . Enviado após handshake e após cada transição de modo.
MODE_ DEBUG _MSG	0x 22	TX	Modo	Payload que informa que o modo ativo é <code>OP_MODE_DEBUG</code> . Enviado após cada transição de modo.

Símbolo	Byte	Direção	Tipo	Descrição
SW_RESET_MSG	0x33	RX	Controle	Comando de reset por software. Processado com prioridade máxima (antes do toggle de modo e da FSM). O STM32 envia [0x90] [0x33] e executa <code>NVIC_SystemReset()</code> após ~50 ms.
ROUTE_A_RECV_MSG	0xDA	RX	FSM	Comando enviado pelo Raspberry Pi ordenando o encaminhamento do objeto para a Rota A. Válido apenas no estado <code>STATE_WAIT_CLASSIFICATION</code> .
ROUTE_B_RECV_MSG	0xDB	RX	FSM	Comando enviado pelo Raspberry Pi ordenando o encaminhamento do objeto para a Rota B. Válido apenas no estado <code>STATE_WAIT_CLASSIFICATION</code> .
OBJ_DETECTED_MSG	0xA0	TX	FSM	Notificação espontânea de que o Sensor 1 detectou um objeto na entrada da esteira (transição <code>IDLE</code> → <code>OBJECT_DETECTED</code>).
CLASS_REQUEST_MSG	0xC0	TX	FSM	Solicitação de classificação enviada ao Raspberry Pi quando o objeto alcança o Sensor 2 (transição <code>OBJECT_DETECTED</code> → <code>WAIT_CLASSIFICATION</code>). O STM32 aguarda 0xDA ou 0xDB em resposta.
ROUTE_A_FORWARD_MSG	0xFA	TX	FSM	Confirmação de que o objeto está sendo encaminhado para a Rota A. Enviado ao receber 0xDA.
ROUTE_B_FORWARD_MSG	0xFB	TX	FSM	Confirmação de que o objeto está sendo encaminhado para a Rota B. Enviado ao receber 0xDB.
ROUTE_A_SUCCESS_DELIVERY_MSG	0xBA	TX	FSM	Notificação de entrega bem-sucedida na Rota A. Disparado quando o Sensor 3 detecta a passagem do objeto. FSM retorna a <code>STATE_IDLE</code> .
ROUTE_B_SUCCESS_DELIVERY_MSG	0xBB	TX	FSM	Notificação de entrega bem-sucedida na Rota B. Disparado quando o Sensor 4 detecta a passagem do objeto. FSM retorna a <code>STATE_IDLE</code> .
DEBUG_MODE_TOGGLE_MSG	0xDD	RX	Modo	Alterna entre <code>OP_MODE_FSM</code> e <code>OP_MODE_DEBUG</code> . Processado com prioridade máxima (após <code>SW_RESET_MSG</code> , antes da FSM) em qualquer modo de operação.

Símbolo	Byte	Direção	Tipo	Descrição
SENS STATUS_ MSG	0x 55	TX	Telemetria	Mensagem espontânea de telemetria dos sensores. Enviada apenas em <code>OP_MODE_DEBUG</code> , somente quando o STATUS_BYTE muda. Frame de 3 bytes: <code>[0x90][0x55][STATUS_BYTE]</code> . Bits 0-3 do STATUS_BYTE indicam o estado de SENS1-SENS4 respectivamente (1 = objeto detectado, 0 = feixe livre). Bits 4-7 reservados (0).
LIGHT EN_M SG	0x E1	RX	Assíncrono (Debug)	Liga o flash/luminária (<code>FLASH_PIN</code> → HIGH). Válido apenas em <code>OP_MODE_DEBUG</code> .
LIGHT DISAB LE_MS G	0x D1	RX	Assíncrono (Debug)	Desliga o flash/luminária (<code>FLASH_PIN</code> → LOW). Válido apenas em <code>OP_MODE_DEBUG</code> .
GATE_ OPEN_ MSG	0x E2	RX	Assíncrono (Debug)	Abre a cancela movendo o servo para <code>openAngle</code> (45°). Válido apenas em <code>OP_MODE_DEBUG</code> .
GATE_ CLOSE_ MSG	0x D2	RX	Assíncrono (Debug)	Fecha a cancela movendo o servo para <code>closeAngle</code> (0°). Válido apenas em <code>OP_MODE_DEBUG</code> .
STPR_ EN_MS G	0x E3	RX	Assíncrono (Debug)	Ativa o motor de passo no modo <i>follow steps</i> (eixo engajado, TIM6 habilitado). Válido apenas em <code>OP_MODE_DEBUG</code> .
STPR DISABL E_MSG	0x D3	RX	Assíncrono (Debug)	Desativa o motor de passo com eixo livre (driver A4988 em SLEEP). Válido apenas em <code>OP_MODE_DEBUG</code> .
SET S TPR FO RWARD _MSG	0x E4	RX	Assíncrono (Debug)	Define o sentido de rotação do motor de passo como frente (<code>stepperDirInst = 0</code>). Válido apenas em <code>OP_MODE_DEBUG</code> .
SET S TPR BA CKWARD _MSG	0x D4	RX	Assíncrono (Debug)	Define o sentido de rotação do motor de passo como trás (<code>stepperDirInst = 1</code>). Válido apenas em <code>OP_MODE_DEBUG</code> .
SET S TPR TG T STPS _MSG	0x E5	RX	Assíncrono (Debug)	Inicia um quadro de 3 bytes. O byte <code>DATA</code> subsequente define a quantidade de passos alvo (<code>targetStepps</code>). É o único frame RX de 3 bytes (<code>[0xAA][0xE5][DATA]</code>). Válido apenas em <code>OP_MODE_DEBUG</code> .

Tabela 2 – Fluxo de Mensagens por Evento

Legenda de quadros completos:

- Quadro TX de sucesso (2 bytes): `[0x90] [PAYLOAD]`
- Quadro TX de telemetria (3 bytes): `[0x90] [0x55] [STATUS_BYTE]`
- Quadro TX de erro (1 byte): `[0x91]`
- Quadro RX padrão (2 bytes): `[0xAA] [CMD]`
- Quadro RX com dado (3 bytes): `[0xAA] [0xE5] [DATA]`

Modo Normal – `OP_MODE_FSM`

Sequência de Inicialização (Handshake)

#	Evento	Agente	Quadro Completo	Bytes (hex)	Descrição
1	Sensores prontos + RPi inicia handshake	RPi → STM32	<code>[START]</code> <code>[SYS_RDY]</code>	AA 10	Raspberry Pi sinaliza que está pronto. STM32 só aceita após todos os sensores medirem 200 Hz.
2	STM32 confirma inicialização	STM32 → RPi	<code>[CMD_OK]</code> <code>[SYS_INIT]</code>	90 01	STM32 confirma sistema inicializado.
3	STM32 informa modo inicial	STM32 → RPi	<code>[CMD_OK]</code> <code>[MODE_FSM]</code>	90 11	STM32 informa que inicia em <code>OP_MODE_FSM</code> . Loop de boot encerra. FSM inicia em <code>STATE_IDLE</code> .

Ciclo Completo – Objeto encaminhado para Rota A

#	Evento / Estado FSM	Agente	Quadro Completo	Bytes (hex)	Descrição
1	<code>IDLE</code> → Sensor 1 detecta objeto	STM32 → RPi	<code>[CMD OK]</code> <code>[OBJ DETECTED]</code>	90 A0	STM32 notifica detecção. FSM avança para <code>STATE_OBJECT_DETECTED</code> .
2	<code>OBJECT_DETECTED</code> → Sensor 2 detecta objeto	STM32 → RPi	<code>[CMD OK]</code> <code>[CLSS_REQUEST]</code>	90 C0	STM32 para o motor e solicita classificação. FSM avança para <code>STATE_WAIT_CLASSIFICATION</code> .
3	RPi define destino: Rota A	RPi → STM32	<code>[START]</code> <code>[ROUTE_A]</code>	AA DA	Raspberry Pi envia o resultado da classificação por visão computacional.
4	STM32 confirma e encaminha para Rota A	STM32 → RPi	<code>[CMD OK]</code> <code>[ROUTE_A_FWD]</code>	90 FA	STM32 confirma, aciona servo (<code>closeAngle</code>), religa motor. FSM avança para <code>STATE_ROUTE_A</code> .

#	Evento / Estado FSM	Agente	Quadro Completo	Bytes (hex)	Descrição
5	<code>ROUTE_A</code> → Sensor 3 detecta objeto	STM32 → RPi	[CMD OK] [ROUTE_A_OK]	90 BA	Entrega confirmada na Rota A. FSM retorna para <code>STATE_IDLE</code> .

Ciclo Completo – Objeto encaminhado para Rota B

#	Evento / Estado FSM	Agente	Quadro Completo	Bytes (hex)	Descrição
1	<code>IDLE</code> → Sensor 1 detecta objeto	STM32 → RPi	[CMD OK] [OBJ DETECTED]	90 A0	STM32 notifica detecção. FSM avança para <code>STATE_OBJECT_DETECTED</code> .
2	<code>OBJECT_DETECTED</code> → Sensor 2 detecta objeto	STM32 → RPi	[CMD OK] [CLSS_REQUEST]	90 C0	STM32 para o motor e solicita classificação. FSM avança para <code>STATE_WAIT_CLASSIFICATION</code> .
3	RPi define destino: Rota B	RPi → STM32	[START] [ROUTE_B]	AA DB	Raspberry Pi envia o resultado da classificação por visão computacional.
4	STM32 confirma e encaminha para Rota B	STM32 → RPi	[CMD OK] [ROUTE_B_FWD]	90 FB	STM32 confirma, aciona servo (<code>openAngle</code>), religa motor. FSM avança para <code>STATE_ROUTE_B</code> .
5	<code>ROUTE_B</code> → Sensor 4 detecta objeto	STM32 → RPi	[CMD OK] [ROUTE_B_OK]	90 BB	Entrega confirmada na Rota B. FSM retorna para <code>STATE_IDLE</code> .

Troca de Modo (qualquer estado)

#	Evento	Agente	Quadro Completo	Bytes (hex)	Descrição
1	RPi solicita entrada em modo debug	RPi → STM32	[START] [DEBUG_TOGGLE]	AA DD	Pode ser enviado em qualquer ponto, em qualquer estado da FSM.
2	STM32 ecoa o comando recebido	STM32 → RPi	[CMD OK] [DEBUG_TOGGLE]	90 DD	Confirmação imediata do recebimento do frame.
3	STM32 informa modo ativo	STM32 → RPi	[CMD OK] [MODE_DEBUG]	90 22	Motor travado (<code>stepperStopEngaged</code>), flash apagado. LED pisca 6× rápido.

#	Evento	Agente	Quadro Completo	Bytes (hex)	Descrição
4	RPi solicita retorno ao modo FSM	RPi → STM32	[START] [DEBUG_TOGGLE]	AA DD	Mesmo comando, reenviado para alternar de volta.
5	STM32 ecoa o comando recebido	STM32 → RPi	[CMD_OK] [DEBUG_TOGGLE]	90 DD	Confirmação imediata do recebimento do frame.
6	STM32 informa modo ativo	STM32 → RPi	[CMD_OK] [MODE_FSM]	90 11	FSM reinicia em <code>STATE_IDLE</code> . LED acende 400 ms e apaga.

Reset por Software (qualquer estado, qualquer modo)

#	Evento	Agente	Quadro Completo	Bytes (hex)	Descrição
1	RPi solicita reset	RPi → STM32	[START] [SW_RESET]	AA 33	Processado com prioridade máxima, antes do toggle de modo e da FSM.
2	STM32 confirma e reinicia	STM32 → RPi	[CMD_OK] [SW_RESET]	90 33	STM32 envia confirmação, aguarda ~50 ms para o frame sair pelo TX e executa <code>NVIC_SystemReset()</code> .

Modo Debug – `OP_MODE_DEBUG`

Em modo debug a FSM fica suspensa. O STM32 responde exclusivamente a comandos assíncronos e envia telemetria de sensores espontaneamente.

Telemetria de Sensores (espontânea)

#	Evento	Agente	Quadro Completo	Bytes (hex)	Descrição
1	STATUS_BYTE dos sensores muda	STM32 → RPi	[CMD_OK] [SENS_STATUS] [STATUS_BYTE]	90 55 NN	Enviado apenas quando o valor do STATUS_BYTE muda em relação ao anterior. NN : bit 0 = SENS1, bit 1 = SENS2, bit 2 = SENS3, bit 3 = SENS4 (1 = objeto detectado, 0 = feixe livre). Bits 4–7 reservados (0).

Comandos Assíncronos

#	Comando enviado	Agente	Quadro Completo	Bytes (hex)	Ação executada pelo STM32	Resposta STM32	Bytes (hex)
1	Liga luminária	RPi → STM32	[START] [LIGHT_EN]	AA E1	FLASH_PIN → HIGH	[CMD OK] [LIGHT_EN]	90 E1
2	Desliga luminária	RPi → STM32	[START] [LIGHT_DIS]	AA D1	FLASH_PIN → LOW	[CMD OK] [LIGHT_DIS]	90 D1
3	Abre cancela	RPi → STM32	[START] [GATE_OPEN]	AA E2	Servo → openAngle (45°)	[CMD OK] [GATE_OPEN]	90 E2
4	Fecha cancela	RPi → STM32	[START] [GATE_CLOSE]	AA D2	Servo → closeAngle (0°)	[CMD OK] [GATE_CLOSE]	90 D2
5	Ativa motor (eixo engajado)	RPi → STM32	[START] [STPR_EN]	AA E3	stepperFollowSteps()	[CMD OK] [STPR_EN]	90 E3
6	Desativa motor (eixo livre)	RPi → STM32	[START] [STPR_DIS]	AA D3	stepperStopDisengaged()	[CMD OK] [STPR_DIS]	90 D3
7	Define direção: frente	RPi → STM32	[START] [STPR_FWD]	AA E4	stepperDirInst = 0	[CMD OK] [STPR_FWD]	90 E4
8	Define direção: trás	RPi → STM32	[START] [STPR_BCK]	AA D4	stepperDirInst = 1	[CMD OK] [STPR_BCK]	90 D4
9	Define quantidade de passos	RPi → STM32	[START] [STPR_TGT [DATA]	AA E5 NN	targetSteps = DATA (NN = valor de 0-255)	[CMD OK] [STPR_TGT]	90 E5
10	Comando desconhecido	RPi → STM32	[START] [???	AA ??	Nenhuma ação	[CMD_ERR]	91

Relatório Parcial — Esteira Separadora (Firmware v3.0)

Sistemas Operacionais Embarcados (SOE) — 2026.1 Autores: Paulo Caleb Fernandes da Silva, Felipe de Castro

1. Descrição do Hardware Controlado

O sistema é composto por uma arquitetura híbrida que divide as responsabilidades entre um microprocessador de alto nível e um microcontrolador dedicado ao controle direto dos dispositivos físicos da esteira.

O **Raspberry Pi 3B** atua como o processador de alto nível, sendo responsável por executar os algoritmos de visão computacional para leitura de QR codes e classificação dos objetos transportados. Uma webcam USB acoplada ao Raspberry Pi captura as imagens no momento adequado do ciclo de operação. As decisões de roteamento são transmitidas ao microcontrolador via comunicação serial UART.

O **STM32G070** é o microcontrolador responsável pelo controle direto dos atuadores e pela leitura dos sensores. Opera a uma frequência de CPU de 64 MHz, obtida por meio do PLL interno a partir do oscilador HSI de 16 MHz (PLLN = 8, PLLM = 1, PLLR = 2). Os periféricos utilizados estão detalhados a seguir.

Timer	Função
TIM15	PWM fixo de 200 Hz para modulação dos lasers
TIM3	Input Capture + DMA para medição de frequência dos 4 sensores
TIM16	PWM de duty variável (50 Hz) para controle do servo motor
TIM6	Base de tempo de 1 µs para controle do motor de passo via interrupção

A **USART2** é configurada a 115200 bps, 8N1, sem controle de fluxo por hardware, para a comunicação serial com o Raspberry Pi 3B.

2. Sensores Laser — Detecção Síncrona por Portadora

Os quatro sensores laser operam sobre o princípio da **detecção síncrona por ausência de portadora**. O TIM15 gera um sinal PWM de 200 Hz com duty cycle de 50% (CCR = 25, ARR = 49, prescaler = 6399 sobre 64 MHz), que modula o acionamento dos transmissores laser. O receptor de cada sensor entrega ao STM32 um sinal digital pulsado nessa mesma frequência enquanto o feixe estiver livre.

A medição é feita pelo **TIM3 em modo Input Capture com DMA** (prescaler = 63, ARR = 65535, base de tempo = 1 µs/tick). Dois valores de captura consecutivos na borda de descida são armazenados em um vetor de dois elementos por DMA. A diferença entre as capturas fornece o período do sinal, e a frequência é calculada como:

$$\text{SENSn_freq} = 1.000.000 / \text{SENSn_capture} \quad [\text{Hz}]$$

24

Quando um objeto interrompe o feixe, a portadora desaparece e a diferença de captura torna-se zero. Nesse caso, `SENSn_flag` é setada em 1 (objeto detectado) e `SENSn_freq` é zerada. O período de amostragem é controlado por `sensUPDT = 10 ms`, que deve ser pelo menos o dobro do período do sinal da portadora (5 ms), garantindo duas bordas capturáveis por ciclo.

A frequência esperada de 200 Hz em todos os quatro sensores é verificada durante o procedimento de inicialização para confirmar que os feixes estão livres e os sensores operacionais antes que o handshake com o Raspberry Pi seja aceito.

3. Servo Motor

O servo motor ES08MA é controlado pelo **TIM16** em modo PWM (prescaler = 63, ARR = 19999, base de tempo = 1 µs/tick, período total = 20 ms = 50 Hz). O mapeamento de ângulo para largura de pulso é realizado pela função `SetServoAngle()`:

```
pulse_length = SERVO_MIN_US + (angle × (SERVO_MAX_US - SERVO_MIN_US) /
SERVO_MAX_ANGLE)
```

onde `SERVO_MIN_US = 600 µs`, `SERVO_MAX_US = 2300 µs` e `SERVO_MAX_ANGLE = 180°`. O valor calculado é carregado diretamente no registrador CCR do TIM16.

Na aplicação, dois ângulos são utilizados: `closeAngle = 0°` (cancela fechada, rota A) e `openAngle = 45°` (cancela aberta, rota B).

4. Motor de Passo

O motor de passo é acionado por um driver **A4988** e controlado pelo **TIM6** (prescaler = 63, ARR variável, base de tempo = 1 µs/tick). A velocidade é ajustada pela função `stepperSetSpeed()`, que recalcula o ARR conforme:

```
TIM6_intFreq = 2 × stepsPerSeconds
newARR = (TIM6_FREQUENCY / TIM6_intFreq) - 1
```

O fator 2 é necessário porque a ISR alterna o pino STEP a cada interrupção (toggle), de modo que um pulso completo (borda de subida + descida) corresponde a dois eventos de interrupção. A velocidade é limitada entre `STEPPER_MIN_VEL = 300 passos/s` e `STEPPER_MAX_VEL = 420 passos/s`.

A ISR `STEPPER_TIM6_ISR()`, chamada pelo callback `HAL_TIM_PeriodElapsedCallback()`, executa a lógica de contagem de passos:

1. Se `stepperEN == 1` e `targetSteps > 0`: alterna o pino STEP; quando `pulse_ctrl == 1` (borda de subida), decrementa `targetSteps`.
2. Se `targetSteps == 0`: força o pino STEP em nível baixo e seta `movement_done_flag = 1`.

A variável `pulse_ctrl` inverte a cada ISR, garantindo que o decremento ocorra somente uma vez por pulso completo.

Três modos de parada estão implementados:

- `stepperStopDisengaged()`: desliga o driver A4988 (pino SLEEP em nível baixo), libera o eixo.
- `stepperStopEngaged()`: mantém o driver energizado (pino SLEEP em nível alto), trava o eixo.
- `stepperFollowSteps()`: habilita o driver, reseta os controles e ativa o TIM6 para retomar a geração de pulsos.

Na FSM, o motor solicita continuamente `stepperMove(350, 0)` (350 passos para frente) nos estados que requerem movimento, garantindo que a esteira não pare enquanto o objeto estiver em trânsito.

5. Protocolo de Comunicação UART

A comunicação entre o STM32 e o Raspberry Pi 3B é realizada por quadros binários estruturados sobre a USART2 (115200 bps, 8N1). A recepção é feita em modo **interrupt-driven** (`HAL_UART_Receive_IT`, 1 byte por vez), processada por uma máquina de estados de três estágios no callback `HAL_UART_RxCpltCallback()`.

5.1 Estrutura dos Quadros

Quadro RX (Raspberry Pi → STM32):

Byte 0	Byte 1	Byte 2 (opcional)
<code>0xAA</code> START	CMD	DATA

O byte DATA só está presente quando `CMD = 0xE5` (`SET_STPR_TGT_STPS_MSG`), que carrega a quantidade de passos como argumento.

Quadro TX (STM32 → Raspberry Pi):

Tipo	Byte 0	Byte 1	Byte 2 (opcional)
Confirmação / Msg	<code>0x90</code>	payload/eco	—
Erro	<code>0x91</code>	—	—
Telemetria	<code>0x90</code>	<code>0x55</code>	STATUS_BYTE

5.2 Tabela de Comandos

Categoria	Byte	Identificador	Direção	Descrição
Handshake	<code>0x10</code>	<code>SYS_RDY_MSG</code>	RBPI3 → STM32	Sinaliza que o RBPI3 está pronto
Handshake	<code>0x01</code>	<code>SYS_INIT_MSG</code>	STM32 → RBPI3	Confirma inicialização do sistema
FSM RX	<code>0xDA</code>	<code>ROUTE_A_RECEIVE_MSG</code>	RBPI3 → STM32	Encaminha objeto para a Rota A

Categoria	Byte	Identificador	Direção	Descrição
FSM RX	0xDB	ROUTE_B_RECEIVE_MSG	RBPI3 → STM32	Encaminha objeto para a Rota B
FSM TX	0xA0	OBJ_DETECTED_MSG	STM32 → RBPI3	Objeto detectado no sensor 1
FSM TX	0xC0	CLSS_REQUEST_MSG	STM32 → RBPI3	Objeto sob a câmera, solicita classificação
FSM TX	0xFA	ROUTE_A_FWRDNG_MSG	STM32 → RBPI3	Confirmação de encaminhamento para Rota A
FSM TX	0xFB	ROUTE_B_FWRDNG_MSG	STM32 → RBPI3	Confirmação de encaminhamento para Rota B
FSM TX	0xBA	ROUTE_A_SCCSS_DLVRY_MSG	STM32 → RBPI3	Entrega confirmada na Rota A
FSM TX	0xBB	ROUTE_B_SCCSS_DLVRY_MSG	STM32 → RBPI3	Entrega confirmada na Rota B
Controle async	0xE1	LIGHT_EN_MSG	RBPI3 → STM32	Liga o flash
Controle async	0xD1	LIGHT_DISABLE_MSG	RBPI3 → STM32	Desliga o flash
Controle async	0xE2	GATE_OPEN_MSG	RBPI3 → STM32	Abre a cancela (servo)
Controle async	0xD2	GATE_CLOSE_MSG	RBPI3 → STM32	Fecha a cancela (servo)
Controle async	0xE3	STPR_EN_MSG	RBPI3 → STM32	Engaja o motor de passo
Controle async	0xD3	STPR_DISABLE_MSG	RBPI3 → STM32	Libera o motor de passo
Controle async	0xE4	SET_STPR_FORWARD_MSG	RBPI3 → STM32	Define direção: frente
Controle async	0xD4	SET_STPR_BACKWARD_MSG	RBPI3 → STM32	Define direção: trás
Controle async	0xE5	SET_STPR_TGT_STPS_MSG	RBPI3 → STM32	Define nº de passos (DATA = valor)
Modo	0xDD	DEBUG_MODE_TOGGLE_MSG	RBPI3 → STM32	Alterna entre modo FSM e Debug
Modo	0x11	MODE_FSM_MSG	STM32 → RBPI3	Informa modo FSM ativo

Categoria	Byte	Identificador	Direção	Descrição
Modo	0x22	MODE_DEBUG_MSG	STM32 → RBPi3	Informa modo Debug ativo
Controle	0x33	SW_RESET_MSG	RBPi3 → STM32	Solicita reset por software (NVIC)
Telemetria	0x55	SENS_STATUS_MSG	STM32 → RBPi3	Envia STATUS_BYTE dos 4 sensores
Resposta	0x90	CMD_OK_MSG	STM32 → RBPi3	Confirmação positiva
Resposta	0x91	CMD_ERR_MSG	STM32 → RBPi3	Erro de reconhecimento

5.3 Parser RX — Máquina de Estados

A recepção de quadros é gerenciada por uma FSM de três estados na ISR de recepção UART:

```

RX_WAIT_START → byte == 0xAA → RX_WAIT_CMD
RX_WAIT_CMD → CMD != 0xE5 → frame completo (2 bytes) → RX_WAIT_START
RX_WAIT_CMD → CMD == 0xE5 → RX_WAIT_DATA
RX_WAIT_DATA → → frame completo (3 bytes) → RX_WAIT_START

```

Qualquer byte fora de frame recebido no estado **RX_WAIT_START** é silenciosamente descartado, protegendo o parser contra ruído ou dados espúrios no barramento.

O frame completo é sinalizado pela flag **rxFrameReady**, com o comando em **rxCMD** e o dado opcional em **rxDATA**.

5.4 Transmissão TX

As funções de transmissão são:

- **sendFrame(status, payload)**: envia 2 bytes [status][payload]. Se **status == 0x91** (CMD_ERR), envia apenas 1 byte.
- **telemetryFrame(status, cmd, data)**: envia 3 bytes [0x90][0x55][STATUS_BYTE].

As mensagens espontâneas da FSM são enviadas por flags no loop principal (ex.: **objDetectedMSG_flag**, **classificationRequestMSG_flag**), evitando chamadas de transmissão dentro da ISR.

5.5 Telemetria de Sensores

A função **sendTelemetryData()** opera exclusivamente no **modo Debug** e monta o **STATUS_BYTE** com as flags dos quatro sensores nos bits 0 a 3:

```

STATUS_BYTE = SENS1_flag | (SENS2_flag << 1) | (SENS3_flag << 2) | (SENS4_flag <<
28

```

3)

A transmissão só ocorre quando o `STATUS_BYTE` muda em relação ao valor anterior (`sensStatusByteLast`), evitando flood no barramento UART. O valor inicial de `sensStatusByteLast` é `0xFF`, forçando o envio na primeira iteração.

6. Modos de Operação

O firmware opera em dois modos selecionáveis pelo comando `DEBUG_MODE_TOGGLE_MSG` (`0xDD`):

Modo	Valor	Descrição
<code>OP_MODE_FSM</code>	0	Modo normal: FSM completa em execução, telemetria desativada
<code>OP_MODE_DEBUG</code>	1	Modo debug: FSM suspensa, apenas comandos assíncronos e telemetria

A troca de modo é processada pela função `handleModeToggle()`, chamada no início de cada iteração do loop principal, com prioridade sobre as demais funções. Ao entrar no modo Debug, o motor é travado (`stepperStopEngaged()`), o flash é apagado e um padrão de pisca no LED de debug confirma a transição. Ao retornar ao modo FSM, o estado é redefinido para `STATE_IDLE` e `last_state` para `STATE_NONE`, forçando a execução das entry actions.

O comando `SW_RESET_MSG` (`0x33`) provoca um reset por software via `NVIC_SystemReset()`, com um delay de 50 ms para garantir que o frame de confirmação seja transmitido antes do reset.

7. Máquina de Estados Finitos (FSM)

A FSM principal da aplicação é implementada na função `FSM()`, chamada no loop principal somente quando `operationMode == OP_MODE_FSM`. O padrão utilizado separa explicitamente as **entry actions** (executadas uma única vez na transição de estado) das **actions de estado** (executadas a cada iteração).

A condição de entrada é verificada por `state != last_state`. Ao executar a entry action, `last_state` é igualado a `state`, fechando a condição.

7.1 Descrição dos Estados

STATE_IDLE (0): Estado inicial e de repouso. O flash é desligado, a cancela é fechada (`SetServoAngle(0)`) e o motor é habilitado em modo follow-steps. A cada iteração, é solicitado um movimento de 350 passos. A transição ocorre quando `SENS1_flag == 1` (objeto detectado na entrada da esteira), momento em que `objDetectedMSG_flag` é setada para notificar o Raspberry Pi.

STATE_OBJECT_DETECTED (1): O motor continua em movimento (350 passos por iteração). A transição ocorre quando `SENS2_flag == 1` (objeto sob a câmera), momento em que `classificationRequestMSG_flag` é setada, solicitando ao Raspberry Pi que realize a leitura do QR code e envie o destino.

STATE_WAIT_CLASSIFICATION (2): O flash é ligado para iluminar a cena de captura e o motor é parado com o eixo travado (`stepperStopEngaged()`). O firmware aguarda o recebimento de `ROUTE_A_RECEIVE_MSG`

(0xDA) ou `ROUTE_B_RECEIVE_MSG` (0xDB). Ao receber o destino, o frame é confirmado com `CMD_OK` e a flag de encaminhamento correspondente é setada.

STATE_ROUTE_A (3): A cancela é mantida fechada (`SetServoAngle(closeAngle)`), o flash é desligado e o motor retoma o movimento. A transição ocorre quando `SENS3_flag == 1` (objeto saiu pela Rota A), setando `routeASuccesDeliveryMSG_flag` e retornando a `STATE_IDLE`.

STATE_ROUTE_B (4): A cancela é aberta (`SetServoAngle(openAngle)`), o flash é desligado e o motor retoma o movimento. A transição ocorre quando `SENS4_flag == 1` (objeto saiu pela Rota B), setando `routeBSuccesDeliveryMSG_flag` e retornando a `STATE_IDLE`.

8. Procedimento de Inicialização e Handshake

Antes de entrar no loop principal, o firmware executa uma rotina de inicialização que bloqueia até que duas condições sejam satisfeitas:

1. **Todos os quatro sensores operacionais:** `SENS1_freq == 200 && SENS2_freq == 200 && SENS3_freq == 200 && SENS4_freq == 200`. Enquanto isso não ocorre, o LED de debug pisca rapidamente (100 ms).
 2. **Handshake com o Raspberry Pi:** o STM32 aguarda receber `[0xAA][0x10]` (`SYS_RDY_MSG`). Após receber os sensores prontos + o handshake, responde com `[0x90][0x01]` (`SYS_INIT_MSG`) seguido de `[0x90][0x11]` (`MODE_FSM_MSG`), informando o modo de operação inicial. Enquanto aguarda o handshake com os sensores já prontos, o LED pisca lentamente (500 ms).
-

9. Controle Assíncrono (Modo Debug)

No modo Debug, a função `handleAsyncCommands()` processa os comandos recebidos para controle individual dos periféricos:

Comando	Ação
<code>LIGHT_EN</code> (0xE1)	Liga o pino <code>FLASH_PIN</code>
<code>LIGHT_DISABLE</code> (0xD1)	Desliga o pino <code>FLASH_PIN</code>
<code>GATE_OPEN</code> (0xE2)	<code>SetServoAngle(openAngle)</code>
<code>GATE_CLOSE</code> (0xD2)	<code>SetServoAngle(closeAngle)</code>
<code>STPR_EN</code> (0xE3)	<code>stepperFollowSteps()</code>
<code>STPR_DISABLE</code> (0xD3)	<code>stepperStopDisengaged()</code>
<code>STPR_FORWARD</code> (0xE4)	Define <code>stepperDirInst = 0</code>
<code>STPR_BACKWARD</code> (0xD4)	Define <code>stepperDirInst = 1</code>
<code>STPR_TGT_STPS</code> (0xE5)	<code>targetStepps = rxDATA</code>

Cada comando recebe uma confirmação `[0x90][CMD_eco]`. Comandos não reconhecidos retornam `[0x91]`.

10. Resumo das GPIOs

Pino	Direção	Função
DIR	Saída	Direção do motor de passo (A4988)
STEP	Saída	Pulsos de passo (A4988)
SLEEP	Saída	Enable/Sleep do driver A4988
dbgLED	Saída	LED de diagnóstico
FLASH_PIN	Saída	Acionamento do flash de iluminação